Jaypee Institute of Information Technology, Sector - 62, Noida

B.Tech CSE II Semester

SDF Lab PBL Report

Real-Time Chat Application

Submitted to

Dr. Amarjeet Prajapati
Dr. Anuja Shukla
Dr. Sulabh Tyagi

Submitted by

Harsh Sharma	2401030232
Karvy Singh	2401030234
Rudra Kumar Singh	2401030237

Letter of Transmittal

Dr. Amarjeet Prajapati
Dr. Anuja Shukla
Dr. Sulabh Tyagi

Department of Computer Science & Information Technology

Jaypee Institute of Information Technology
Sector 62, Noida

Subject: Submission of Report on "Real-Time Chat Application"

Respected Sir,

We are pleased to submit our report for our project *"Real-Time Chat Application"* as part of our coursework. This report documents our work throughout the project's lifetime.

This project is a Real-Time Desktop Chat Application developed in C++ using the Qt framework for the GUI, Boost.Asio for networking, and SQLite for local data storage. It follows an object-oriented approach and supports real-time text messaging, user authentication, and chat history persistence. The system includes a client with a login and chat interface, and a server that handles connections, user management, and message routing. It demonstrates advanced C++ concepts like multithreading, inheritance, virtual functions, and integrates SQL for data handling.

We have endeavoured to apply all topics covered in SDF-II and hope that this meets your expectations.

Thank you for your guidance and the opportunity to work on this project.

Sincerely,

Harsh Sharma (2401030232)
Karvy Singh (2401030234)
Rudra Kumar Singh (2401030237)

Date: May 3, 2025

Contents

1 Introduction

1.1 Background and Context

In the modern era of digital communication, real-time messaging systems have become essential tools for both personal and professional interactions. Traditional communication methods, such as email or SMS, often fail to provide the immediacy and interactivity required in fast-paced environments. As a result, instant messaging platforms have gained widespread popularity for enabling users to exchange messages, files, and multimedia content instantly over a network.

This project stems from the growing need for a **lightweight, desktop-based chat solution** that prioritizes real-time responsiveness, user-friendly interface design, and secure data management. Built using **C++** with the **Qt framework** for GUI and **Boost.Asio** for networking, the application also incorporates a **SQLite** database to ensure persistent message storage. This context emphasizes the technical and practical importance of the project in delivering a fully-functional, modular, and extensible communication system.

1.2 Problem Statement

Most existing chat applications are either mobile-focused, cloud-dependent, or lack customization for desktop use. Users often have limited control over data, real-time performance is inconsistent, and many tools are not designed for simple local deployment or extensibility. There is a need for a lightweight, desktop-based chat system that provides real-time communication, secure local data handling, and a clean, user-friendly interface—without relying on third-party servers or bloated software stacks.

1.3 Project Objectives

The primary objective of this project is to design and implement a desktop-based real-time chat application using C++. The key goals include:

- Enable real-time text communication between authenticated users.
- Build a responsive and intuitive graphical user interface using Qt.
- Implement a secure, local authentication system using SQLite.
- Store and retrieve message history efficiently using a local database.
- Ensure modular, scalable, and maintainable code using object-oriented principles.
- Support future enhancements like profile management, notifications, or group chats.

1.4 Scope of the Project

This project focuses on building a desktop chat application with core features required for real-time communication. The current scope includes:

- User Authentication: Login system with local credential verification using SQLite.

- Real-Time Messaging: Instant text message exchange over a custom client-server protocol using Boost.Asio.
- Graphical User Interface: Desktop GUI built with Qt for ease of use and interaction.
- Message Storage: Local persistence of chat history per user using SQLite.
- OOP-Based Architecture: Implementation using object-oriented design for better maintainability and extensibility.

The scope does not include mobile support, end-to-end encryption, media file transfer, or cloud-based deployment in this version. However, the system is designed to allow easy integration of these features in future iterations.

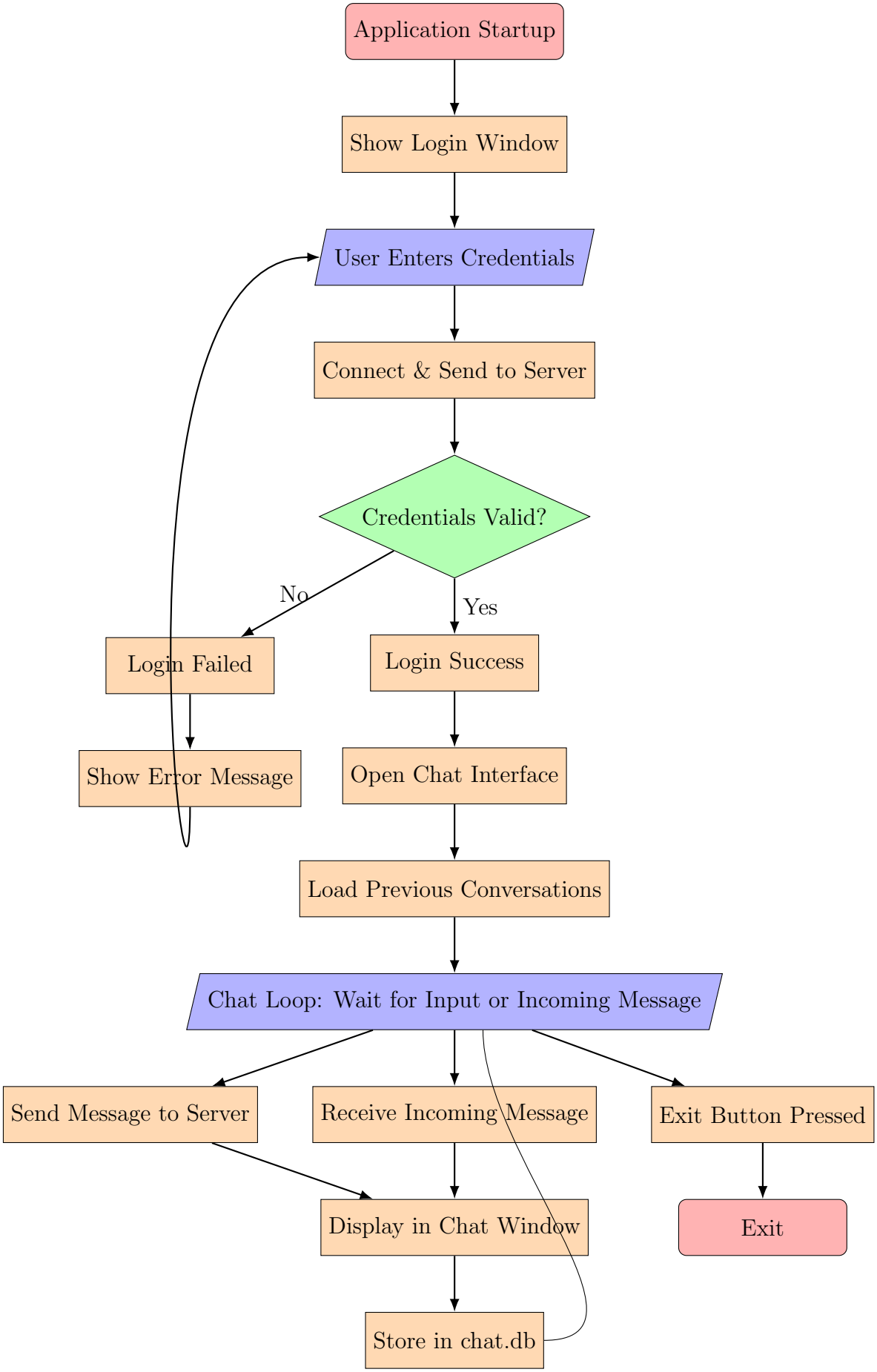
2 Architecture and Design

2.1 Overview

The Real-Time Chat Application is based on a modular, client-server architecture tailored for desktop systems. Implemented in `C++`, it integrates libraries like `Qt` for the graphical user interface, `Boost.Asio` for asynchronous networking, and `SQLite` for local data persistence. The system design emphasizes object-oriented principles, aiming for scalability, clarity, and extensibility.

2.2 Flowchart

A flowchart depicting the application's control flow is presented on the following page.


Application Startup
Show Login Window
User Enters Credentials
Connect & Send to Server
Credentials Valid?
No
Yes
Login Failed
Login Success
Show Error Message
Open Chat Interface
Load Previous Conversations
Chat Loop: Wait for Input or Incoming Message
Send Message to Server
Receive Incoming Message
Exit Button Pressed
Display in Chat Window
Exit
Store in chat.db

2.3 System Architecture

2.3.1 Client Side

The client is a standalone desktop application responsible for user interaction and communication with the server. Key roles of the client include:

- Managing login and authentication workflows
- Displaying a real-time messaging interface
- Sending and receiving messages asynchronously
- Storing and retrieving chat history from a local SQLite database

2.3.2 Server Side

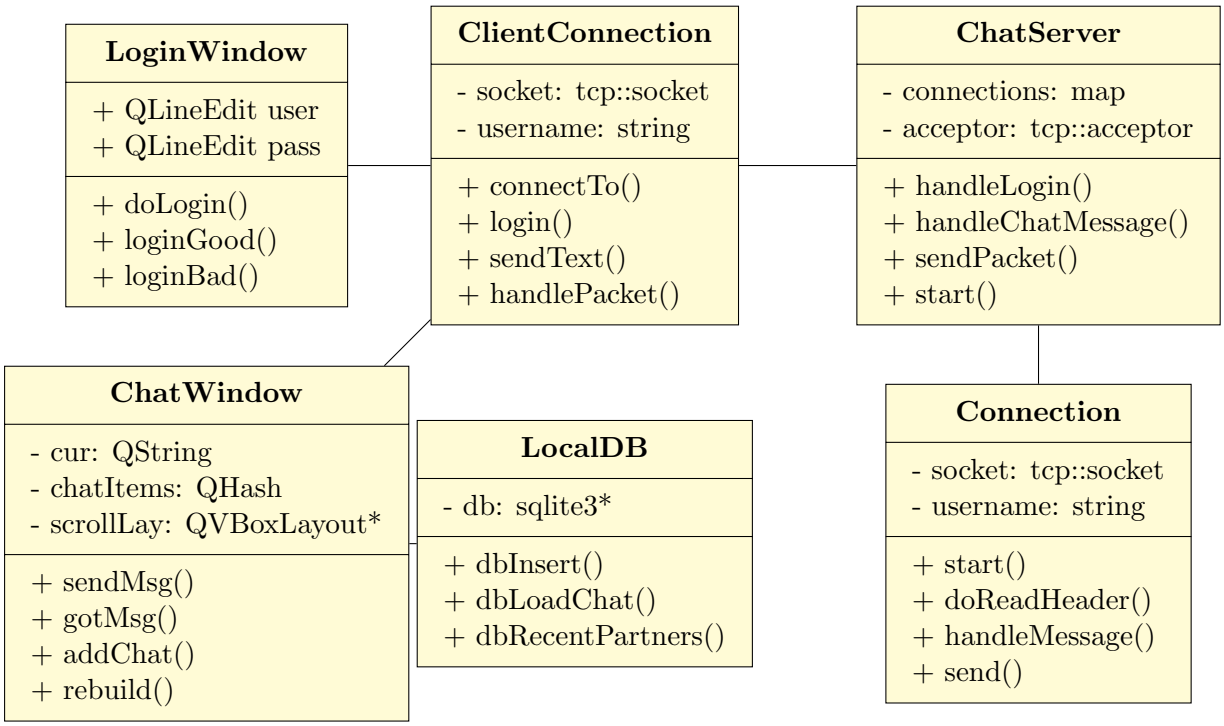
The server acts as a central hub, managing user sessions and routing messages between clients. It handles:

- Verifying user credentials through a persistent SQLite database
- Keeping track of active user sessions
- Relaying messages to recipients or notifying senders of delivery issues

2.4 Component Design

- **LoginWindow:** A Qt widget that allows the user to input credentials and connect to the server.
- **ClientConnection:** Manages TCP socket communication, handles protocol encoding/decoding, and emits signals for UI updates.
- **ChatWindow:** The main interface for messaging. It supports chat switching, message entry, and real-time display.
- **Local Database Layer:** Stores chat history and recent contacts using `SQLite`.
- **ChatServer:** Accepts client connections, manages sessions, and delegates communication logic.
- **Connection:** Represents individual client sessions, handles message processing and disconnection logic.

2.5 Class Diagram



2.6 Communication Protocol

A custom binary protocol ensures fast and structured communication between clients and the server. The format includes:

- **4 bytes:** Magic header ("J", "I", "I", "T")
- **1 byte:** Packet type (e.g., login = `0x01`, message = `0x02`)
- **4 bytes:** Payload length in big-endian format
- **N bytes:** JSON-encoded message payload

This hybrid format combines binary framing (for performance) and JSON (for flexibility), allowing the protocol to evolve without breaking compatibility.

2.7 Design Principles and Patterns

The application follows established object-oriented practices:

- **Encapsulation:** Separation of UI, networking, and storage logic into independent modules.
- **Event-Driven Architecture:** Implemented using Qt's signal-slot mechanism for responsive UIs.
- **Memory Safety:** Server components use smart pointers (`std::shared_ptr`) for lifetime management.
- **Efficient Data Structures:** STL containers such as `std::vector`, `std::map`, and `QString` simplify internal data handling.

2.8 Flow Control

The flow of the application is reactive and asynchronous, designed to keep the interface responsive while continuously managing communication.

On the **client side**, the application begins by displaying a login window. Upon entering credentials, a connection is made to the server using Boost.Asio. If authentication is successful, control moves to the main chat interface.

At this point, the system enters a continuous loop where it:

1. Waits for user input to send a message.
2. Listens for incoming messages from the server.
3. Updates the chat display accordingly and stores the data locally.

These operations are performed on separate threads: Qt handles the GUI thread, while Boost.Asio operates in the background for network I/O. This ensures that incoming messages can be processed even while the user is typing or interacting with the interface.

On the **server side**, asynchronous callbacks are used to accept new connections, parse incoming packets, and respond appropriately. Once a client sends a login or message packet, the server handles it based on its type and either forwards it or replies with an appropriate notification.

This flow architecture makes the application highly responsive and capable of handling multiple concurrent users without blocking or UI delays. The separation of concerns between GUI, networking, and storage also makes the system easier to debug, extend, and maintain.

3 Screenshots

This section presents screenshots of key components of the application interface, demonstrating its functionality and user experience.

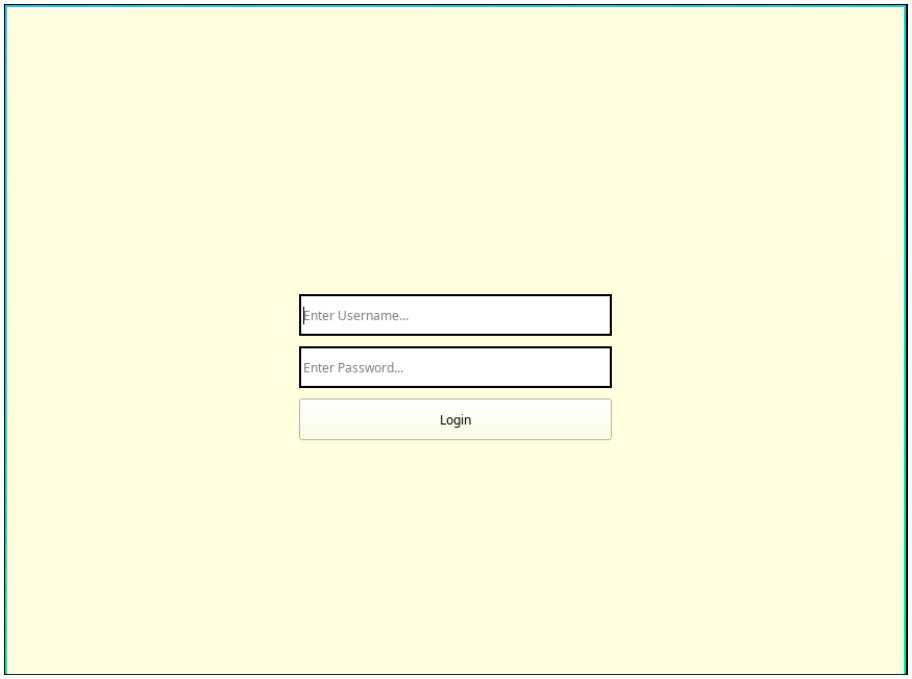


Figure 1: Login Window – User enters credentials to connect to the server.

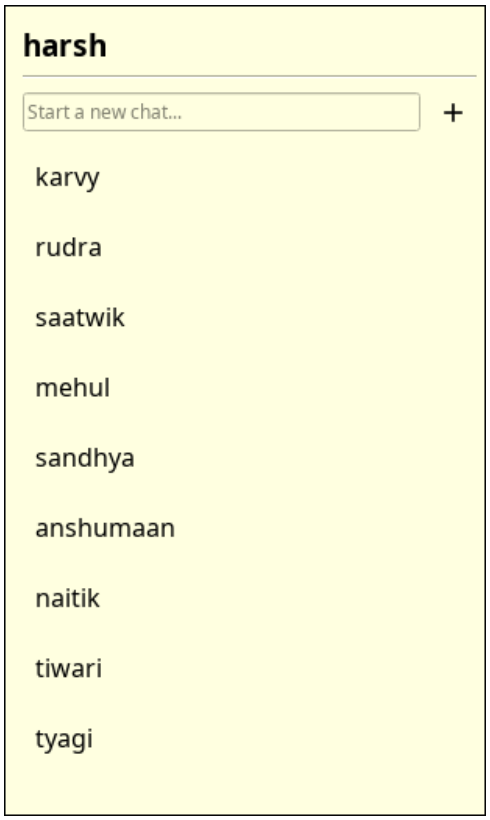


Figure 2: Contacts – List showing previous conversations.

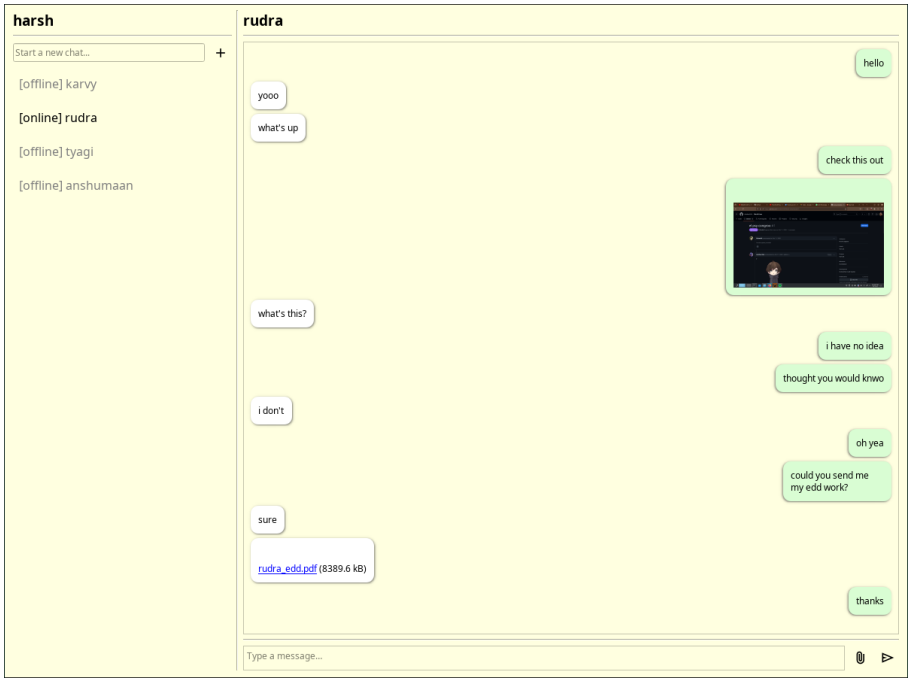


Figure 3: Chat Interface – Real-time messaging window with contacts and message bubbles.

4 Conclusion

4.1 Summary

This project successfully demonstrates the design and implementation of a real-time desktop chat application using C++. The application provides core features such as user authentication, real-time text messaging, persistent local storage of chat history, and a clean graphical interface.

By employing modern C++ libraries like Qt for GUI design and Boost.Asio for asynchronous networking, along with a lightweight SQLite backend, the system maintains modularity, efficiency, and responsiveness. The project also reinforces object-oriented principles and multi-threaded event handling in a practical context.

4.2 Future Work and Recommendations

While the current version meets the core requirements of a real-time desktop chat application, several enhancements are envisioned to improve functionality and user experience:

- **End-to-End Encryption:** Implementing secure encryption to protect message contents during transmission.
- **Group Chat Functionality:** Enabling users to create and participate in multi-user chat groups.
- **Forward Messages:** Allowing users to forward previously received messages to other users or groups.
- **Typing Indicators:** Showing real-time feedback when a user is composing a message.

- **Voice Messages:** Supporting the recording and sending of short audio clips.
- **Voice and Video Calling:** Adding real-time audio and video communication between users.
- **Status Messages:** Enabling users to set custom status messages visible to others (e.g., busy, available).
- **Block Feature:** Allowing users to block specific contacts from messaging or viewing their status.

Continued refinement of the codebase, proper unit testing, and integration of deployment tools would also contribute to making the application production-ready.

5 References

[R1] Qt Framework Documentation – `https://www.qt.io`

[R2] Boost.Asio Networking Library – `https://www.boost.org/doc/libs/release/doc/html/boost_asio.html`

[R3] SQLite Database Engine – `https://www.sqlite.org/index.html`

[R4] C++ STL Reference (cppreference) – `https://en.cppreference.com/w/`

[R5] nlohmann/json (Modern C++ JSON) – `https://nlohmann.github.io/json/`

[R6] Qt Signals and Slots Documentation – `https://doc.qt.io/qt-6/signalsandslots.html`